PROJECT SUBMISSION

MASTERMIND CODEBREAKER GAME

CSC311

Project No: 20

Name	Satyaki Banerjee
Roll	CSE/25082/1482
Registration No.	1482
Group No.	6

Submitted for Academic Evaluation

2026

INTRODUCTION

The Mastermind Codebreaker Game is a MIPS/QtSPIM-based digit sequence guessing game. The program generates a secret four-digit sequence and allows the player up to seven attempts to identify it. After each guess, the program provides feedback by reporting the number of digits in the correct position, the number of correct digits in the wrong position, and the number of digits that are not present. The project demonstrates the practical use of MIPS assembly programming concepts such as registers, system calls, loops, conditional branching, arrays, procedures, arithmetic operations, and stack usage.

AIM

To design and implement a four-digit sequence guessing game in MIPS assembly using QtSPIM, where the program generates a secret sequence, accepts and validates user guesses, compares each guess with the secret, provides meaningful feedback, limits the player to seven attempts, and displays the final game result.

MIPS / QtSPIM Source Code

```
# ============================================================
# DIGIT SEQUENCE GUESSING GAME
# MIPS / QtSPIM
#
# Secret sequence : 4 digits
# Attempts        : 7
#
# Feedback:
#   Correct position
#   Correct digit, wrong position
#   Digit not present
# ============================================================

.data

title:          .asciiz "\n===== DIGIT SEQUENCE GUESSING GAME =====\n"
rules:          .asciiz "Guess the 4-digit secret sequence.\n"
rules2:         .asciiz "Each digit is between 0 and 9.\n"
rules3:         .asciiz "You have 7 attempts.\n\n"

attemptMsg:     .asciiz "\nAttempt "
ofMsg:          .asciiz " of 7\n"

inputMsg:       .asciiz "Enter your 4-digit guess: "

invalidMsg:     .asciiz "\nInvalid input! Enter exactly 4 digits.\n"

correctMsg:     .asciiz "Correct positions: "
wrongPosMsg:    .asciiz "\nCorrect digits, wrong positions: "
notPresentMsg:  .asciiz "\nDigits not present: "

winMsg:         .asciiz "\n\n*** CONGRATULATIONS! YOU WON! ***\n"
loseMsg:        .asciiz "\n\n*** GAME OVER! ***\n"
secretMsg:      .asciiz "The secret sequence was: "

newline:        .asciiz "\n"

# Space for secret and guess
secret:         .space 16
guess:          .space 16

.text
.globl main

# ============================================================
# MAIN PROGRAM
# ============================================================

main:

    # ---------------------------------------------------------
    # Display title and rules
    # ---------------------------------------------------------

    li $v0, 4
    la $a0, title
    syscall

    li $v0, 4
    la $a0, rules
    syscall

    li $v0, 4
    la $a0, rules2
    syscall

    li $v0, 4
    la $a0, rules3
    syscall


    # ---------------------------------------------------------
    # Generate secret sequence
    # ---------------------------------------------------------

    jal generate_secret


    # ---------------------------------------------------------
    # Initialize attempt counter
    # ---------------------------------------------------------

    li $s0, 1              # s0 = current attempt
    li $s1, 7              # s1 = maximum attempts


# ============================================================
# GAME LOOP
# ============================================================
game_loop:
```

```
# Check whether attempts are finished

bgt $s0, $s1, game_over


# ---------------------------------------------------------
# Print attempt number
# ---------------------------------------------------------

li $v0, 4
la $a0, attemptMsg
syscall

li $v0, 1
move $a0, $s0
syscall

li $v0, 4
la $a0, ofMsg
syscall


# ---------------------------------------------------------
# Ask user for guess
# ---------------------------------------------------------

li $v0, 4
la $a0, inputMsg
syscall

li $v0, 5
syscall

move $t0, $v0


# ---------------------------------------------------------
# Convert integer into four digits
#
# Example:
#
# User enters 1234
#
# thousands = 1
# hundreds  = 2
# tens      = 3
# units     = 4
# ---------------------------------------------------------

# Check range: 1000 - 9999

li $t1, 1000
blt $t0, $t1, invalid_input

li $t1, 9999
bgt $t0, $t1, invalid_input


# Address of guess array

la $t2, guess


# -------------------------
# Extract thousands digit
# -------------------------

li $t1, 1000
div $t0, $t1
mflo $t3

sb $t3, 0($t2)


# Remove thousands digit

li $t1, 1000
mul $t4, $t3, $t1
sub $t0, $t0, $t4


# -------------------------
# Extract hundreds digit
# -------------------------

li $t1, 100
div $t0, $t1
mflo $t3

sb $t3, 1($t2)


# Remove hundreds digit

li $t1, 100
```

```
    mul $t4, $t3, $t1
    sub $t0, $t0, $t4


    # -------------------------
    # Extract tens digit
    # -------------------------

    li $t1, 10
    div $t0, $t1
    mflo $t3

    sb $t3, 2($t2)


    # -------------------------
    # Units digit
    # -------------------------

    sb $t0, 3($t2)


    # ---------------------------------------------------------
    # Compare guess with secret
    # ---------------------------------------------------------

    jal compare_guess

    # compare_guess returns:
    #
    # v0 = correct positions
    # v1 = correct digits in wrong positions
    # v2 = digits not present


    # ---------------------------------------------------------
    # Display feedback
    # ---------------------------------------------------------

    li $v0, 4
    la $a0, correctMsg
    syscall

    li $v0, 1
    move $a0, $v0

    # The value was overwritten above, so retrieve result
    # from saved registers used by compare_guess.

    move $a0, $s2
    li $v0, 1
    syscall


    li $v0, 4
    la $a0, wrongPosMsg
    syscall

    move $a0, $s3
    li $v0, 1
    syscall


    li $v0, 4
    la $a0, notPresentMsg
    syscall

    move $a0, $s4
    li $v0, 1
    syscall


    li $v0, 4
    la $a0, newline
    syscall


    # ---------------------------------------------------------
    # Check if player won
    # ---------------------------------------------------------

    li $t0, 4

    beq $s2, $t0, player_won


    # ---------------------------------------------------------
    # Increase attempt counter
    # ---------------------------------------------------------

    addi $s0, $s0, 1

    j game_loop
# ============================================================
# INVALID INPUT
```

```
# ============================================================

invalid_input:

    li $v0, 4
    la $a0, invalidMsg
    syscall

    j game_loop


# ============================================================
# PLAYER WON
# ============================================================

player_won:

    li $v0, 4
    la $a0, winMsg
    syscall

    j exit_program


# ============================================================
# GAME OVER
# ============================================================

game_over:

    li $v0, 4
    la $a0, loseMsg
    syscall

    li $v0, 4
    la $a0, secretMsg
    syscall


    # Print secret sequence

    la $t0, secret
    li $t1, 0


print_secret:

    beq $t1, 4, finish_secret

    lb $t2, 0($t0)

    li $v0, 1
    move $a0, $t2
    syscall

    addi $t0, $t0, 1
    addi $t1, $t1, 1

    j print_secret


finish_secret:

    li $v0, 4
    la $a0, newline
    syscall


# ============================================================
# EXIT
# ============================================================

exit_program:

    li $v0, 10
    syscall


# ============================================================
# GENERATE SECRET
# ============================================================
#
# Uses system time as a changing value.
#
# syscall 30:
#   returns current time in milliseconds
#
# We repeatedly perform:
#
#       seed = seed * 1103515245 + 12345
#
# This creates pseudo-random values.
#
# Each digit:
#
```

```
#       digit = seed % 10
#
# ============================================================

generate_secret:

    # Get system time

    li $v0, 30
    syscall

    move $t0, $a0


    la $t1, secret

    li $t2, 0


generate_loop:

    beq $t2, 4, generate_done


    # Linear congruential generator

    li $t3, 1103515245
    multu $t0, $t3
    mflo $t0

    li $t3, 12345
    addu $t0, $t0, $t3


    # Get digit 0-9

    li $t4, 10
    divu $t0, $t4
    mfhi $t5


    # Store digit

    sb $t5, 0($t1)


    addi $t1, $t1, 1
    addi $t2, $t2, 1

    j generate_loop


generate_done:

    jr $ra


# ============================================================
# COMPARE GUESS
# ============================================================
#
# This procedure calculates:
#
# 1. Correct positions
# 2. Correct digits but wrong positions
# 3. Digits not present
#
# ============================================================

compare_guess:

    # ----------------------------------------------------------
    # Save return address
    # ----------------------------------------------------------

    addi $sp, $sp, -4
    sw $ra, 0($sp)


    # ----------------------------------------------------------
    # Initialize counters
    # ----------------------------------------------------------

    li $s2, 0              # correct position
    li $s3, 0              # wrong position
    li $s4, 0              # not present


    # ==========================================================
    # STEP 1
    # Check exact positions
    # ==========================================================

    la $t0, secret
    la $t1, guess
```

```
    li $t2, 0


exact_loop:

    beq $t2, 4, exact_done


    lb $t3, 0($t0)
    lb $t4, 0($t1)


    # Compare secret[i] and guess[i]

    beq $t3, $t4, exact_match


    # Not exact

    addi $t0, $t0, 1
    addi $t1, $t1, 1
    addi $t2, $t2, 1

    j exact_loop


exact_match:

    addi $s2, $s2, 1

    addi $t0, $t0, 1
    addi $t1, $t1, 1
    addi $t2, $t2, 1

    j exact_loop


exact_done:


    # ========================================================
    # STEP 2
    # Check digits occurring anywhere in secret
    #
    # We count matching digit values.
    # Exact matches are excluded because we already counted
    # them.
    # ========================================================

    la $t0, guess
    li $t1, 0


wrong_position_outer:

    beq $t1, 4, wrong_position_done


    # Get current guessed digit

    lb $t2, 0($t0)


    # Search in secret

    la $t3, secret
    li $t4, 0


wrong_position_inner:

    beq $t4, 4, next_guess_digit


    lb $t5, 0($t3)


    # Compare guess digit with secret digit

    bne $t2, $t5, no_match


    # --------------------------------------------------------
    # Check whether this was an exact position
    # --------------------------------------------------------

    # If guess index == secret index,
    # it was already counted as correct position.

    beq $t1, $t4, no_match


    # Otherwise it is a wrong-position match.

    addi $s3, $s3, 1
```

```
    j next_guess_digit


no_match:

    addi $t3, $t3, 1
    addi $t4, $t4, 1

    j wrong_position_inner


next_guess_digit:

    addi $t0, $t0, 1
    addi $t1, $t1, 1

    j wrong_position_outer


wrong_position_done:


    # ========================================================
    # STEP 3
    # Calculate digits not present
    # ========================================================

    # Total digits = 4
    #
    # not present =
    # 4 - correct positions - wrong positions
    #
    # --------------------------------------------------------

    li $t0, 4

    sub $s4, $t0, $s2
    sub $s4, $s4, $s3


    # --------------------------------------------------------
    # Restore return address
    # --------------------------------------------------------

    lw $ra, 0($sp)
    addi $sp, $sp, 4

    jr $ra
```

PROGRAM EXECUTION RESULT

The MIPS/QtSPIM Digit Sequence Guessing Game was executed successfully. The program displayed the game rules, accepted seven guesses, provided feedback for each attempt, and displayed the secret sequence when the maximum number of attempts was reached without a successful guess.

Observed Execution

Secret sequence: 6121
Total allowed attempts: 7
Attempts used: 7
Final status: Game Over

Execution Summary

Attempt	Guess	Correct Position	Wrong Position	Not Present
1	2345	0	1	3
2	2356	0	1	3
3	5647	0	1	3
4	5214	0	2	2
5	8563	0	1	3
6	5874	0	0	4
7	4129	2	1	1

Result

The program completed all seven attempts and correctly reached the **GAME OVER** condition. The secret sequence generated during this run was **6121**, which was revealed at the end of the execution. The feedback system reported the number of digits in the correct position, digits present but in the wrong position, and digits not present.

Conclusion

The Digit Sequence Guessing Game demonstrates the use of MIPS assembly programming concepts including system calls, registers, arrays, loops, conditional branches, procedures, stack-based return-address storage, arithmetic operations, pseudo-random secret generation, and comparison logic. The observed run shows that the program accepts guesses, evaluates them, updates the attempt count, and terminates with the appropriate game result.

Execution Screenshot

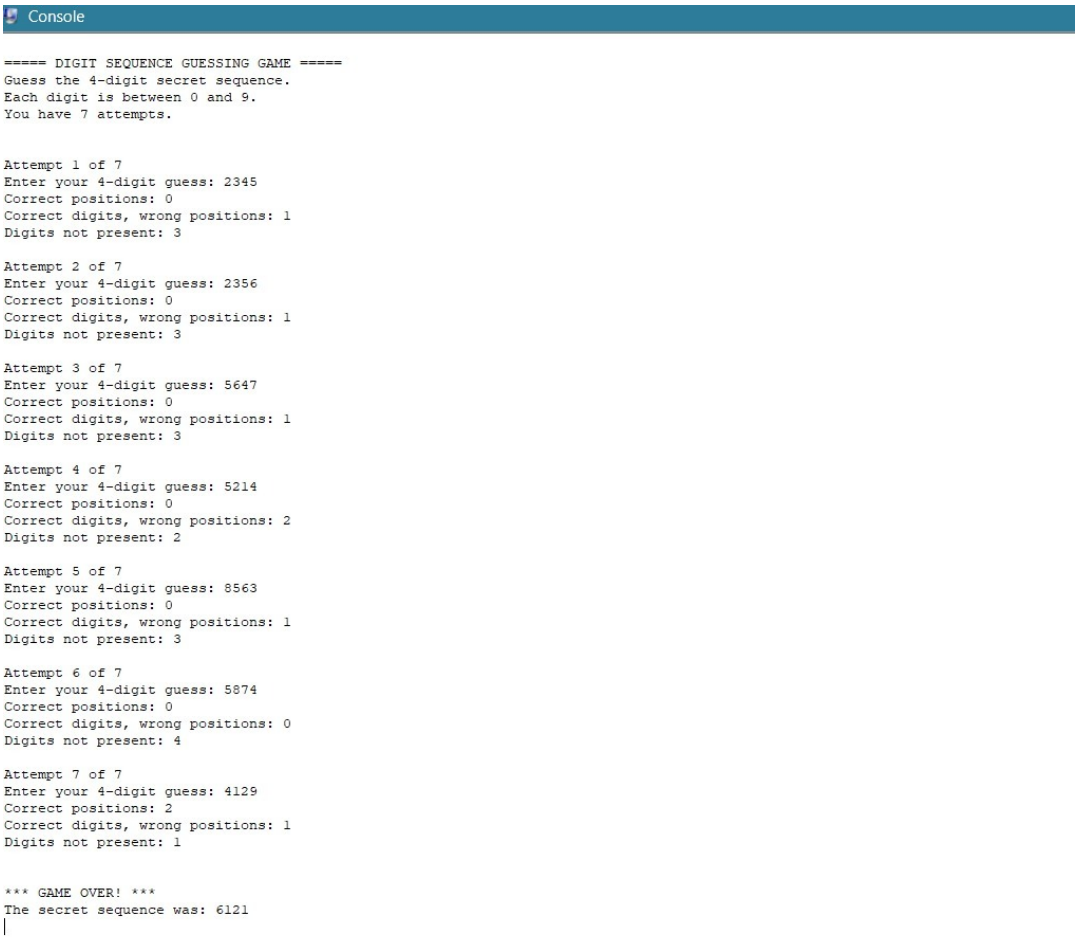


Figure 1: QtSPIM console output showing seven attempts and the final secret sequence 6121.